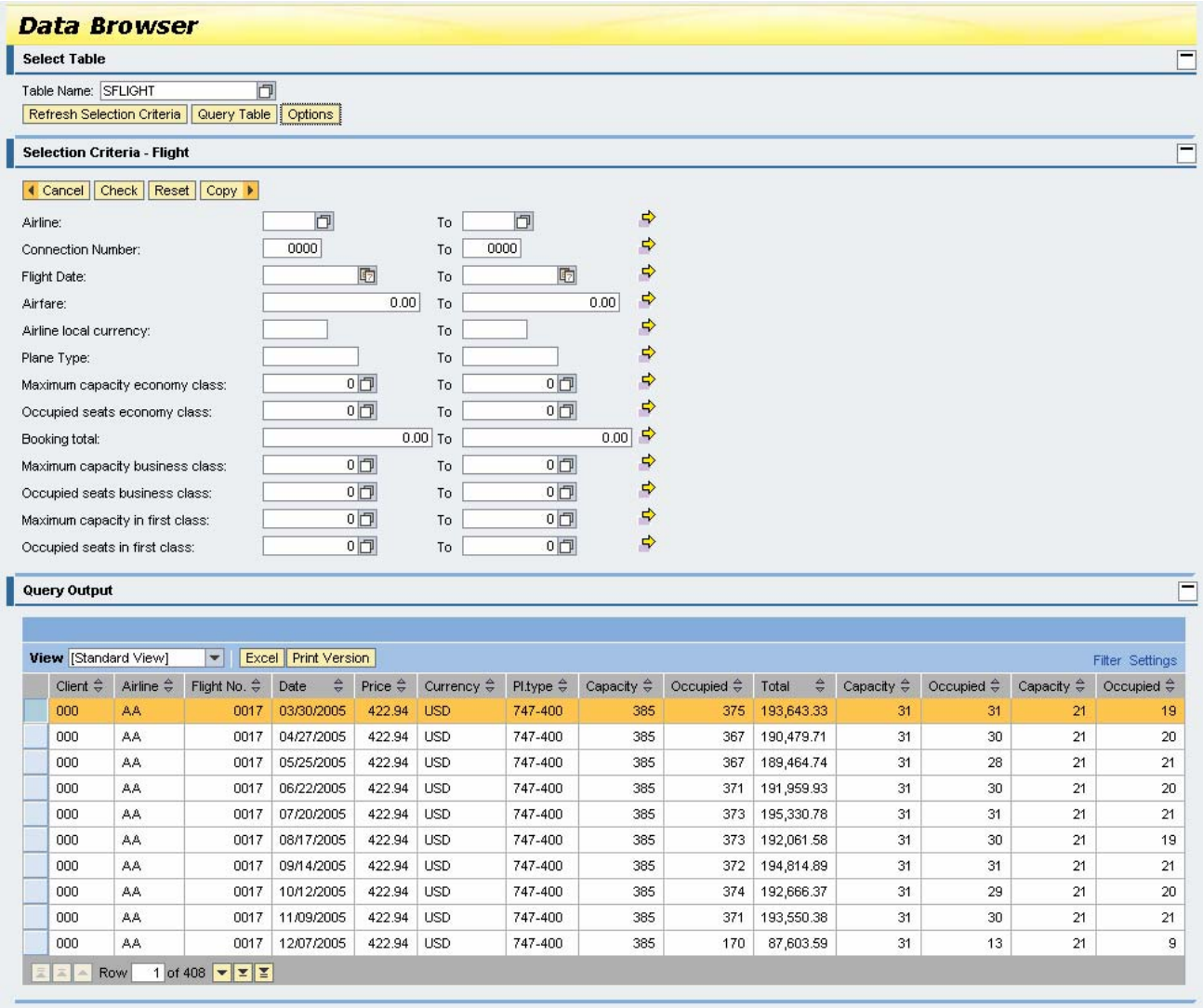
Web Dynpro for ABAP: Recreate the SE16 Data Browser

Applies To:

This tutorial applies to Web Dynpro for ABAP. This technology is available in SAP NetWeaver 04s Application Server - ABAP. An evaluation version of this release can be downloaded from SDN for registered members.

Summary

The purpose of this tutorial is to show several aspects of advanced Web Dynpro programming by rebuilding the classic SE16 transaction with Web Dynpro. The end result is a generic Data Browser that allows a robust query mechanism against nearly any transparent table in the ABAP database.



Client	Airline	Flight No.	Date	Price	Currency	Pl.type	Capacity	Occupied	Total	Capacity	Occupied	Capacity	Occupied
000	AA	0017	03/30/2005	422.94	USD	747-400	385	375	193,643.33	31	31	21	19
000	AA	0017	04/27/2005	422.94	USD	747-400	385	367	190,479.71	31	30	21	20
000	AA	0017	05/25/2005	422.94	USD	747-400	385	367	189,464.74	31	28	21	21
000	AA	0017	06/22/2005	422.94	USD	747-400	385	371	191,959.93	31	30	21	20
000	AA	0017	07/20/2005	422.94	USD	747-400	385	373	195,330.78	31	31	21	21
000	AA	0017	08/17/2005	422.94	USD	747-400	385	373	192,061.58	31	30	21	19
000	AA	0017	09/14/2005	422.94	USD	747-400	385	372	194,814.89	31	31	21	21
000	AA	0017	10/12/2005	422.94	USD	747-400	385	374	192,666.37	31	29	21	20
000	AA	0017	11/09/2005	422.94	USD	747-400	385	371	193,550.38	31	30	21	21
000	AA	0017	12/07/2005	422.94	USD	747-400	385	170	87,603.59	31	13	21	9

Web Dynpro for ABAP: Recreate the SE16 Data Browser

There are several key Web Dynpro ABAP technologies that will be introduced or expanded upon in this tutorial:

- Dynamic UI elements and programming
- Select-Options for Search Criteria
- ALV for Query Output
- Usage of the Message Manager for the display of Error Messages

By: Thomas Jung

Company: SAP Labs, LLC.

Date: 09 February 2006

Table of Contents

Applies To:....1

Summary....1

Table of Contents....2

Web Dynpro Component....3

Component Controller....4

View....5

- View Context....5
- Actions....6
- Layout....6
  - TABLE_TRAY_CP....7
  - OPTIONS_TRAY....8
  - SELECTIONS_TRAY....9
  - OUTPUT_TRAY....10
- View Attributes....10
- Methods....10
  - Method WDDOINIT....11
  - Method ONACTIONOPTIONS_SEL....12
  - Method ONACTIONREFRESH_SEL....13

Method ONACTIONRUN_QUERY 14

Method AUTHORITY_CHECK 18

Method BUILD_SELECT_OPTIONS 20

Method TABLENAME_VALIDATION 22

Windows 23

Web Dynpro Application 24

Author Bio 25

Disclaimer & Liability Notice 25

Web Dynpro Component

We will start by creating a new component, which I named Z_SE16_WDA. During the creation dialog, you will also want to create a window, which I named MAIN.

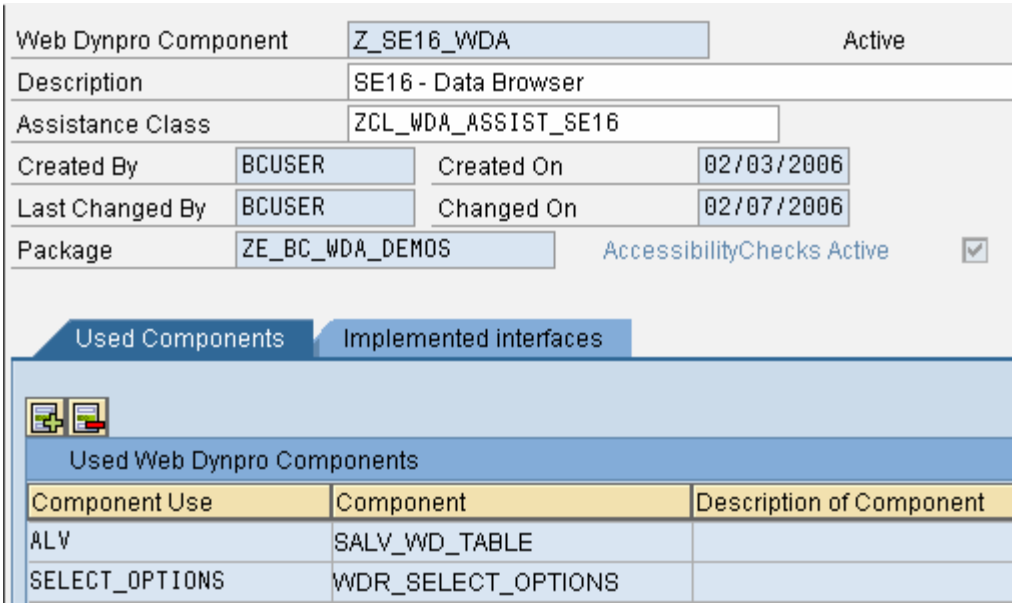


Figure 1 – Web Dynpro Component

There are two activities that must take place in the Change Component screen. First we must add the Web Dynpro Components that we will refer to inside our component. Under "Used Web Dynpro Components" add the two entries seen in Figure 1. This will allow us to use the ALV and Select Options components later.

We will also later be using some texts for our error messages. We want these texts to be translatable. For this we will want to use ABAP text elements. In order to support text elements within our Web Dynpro Component, we must create an assistance class. Simply type the name of the class that you want to create into the Assistance Class field and hit enter. A dialog will ask you if you wish to generate the class in question.

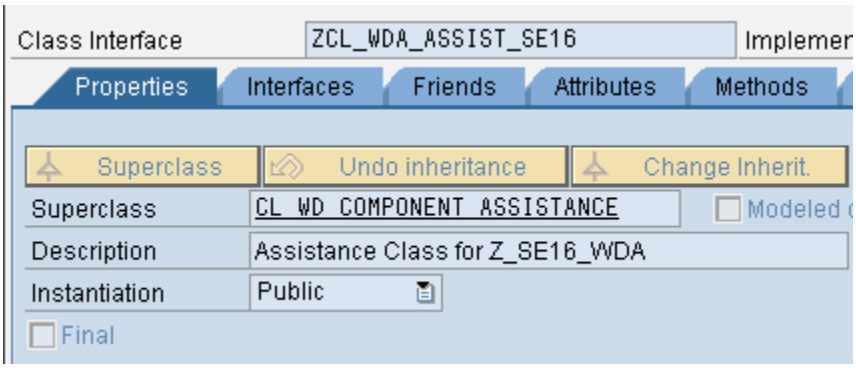


Figure 2 - Assistance Class

Although the generation routine takes care of the details of creating the Assistance Class, it should have an inheritance from the superclass CL_WD_COMPONENT_ASSISTANCE. Later in our coding, as the below sample shows, we will be able to reference text elements from this assistance class as though they were local to the Web Dynpro Component itself.

```
l_message_manager->report_attribute_error_message(
      message_text   = 'Please supply a Table Name'(e01)
      element        = elem_input
      attribute_name = 'TABLENAME' ).
```

While you are looking at the assistance class, we might as well go ahead and fill in the text elements that we will need later. From the Assistance Class Maintenance choose menu Goto->Text Elements.

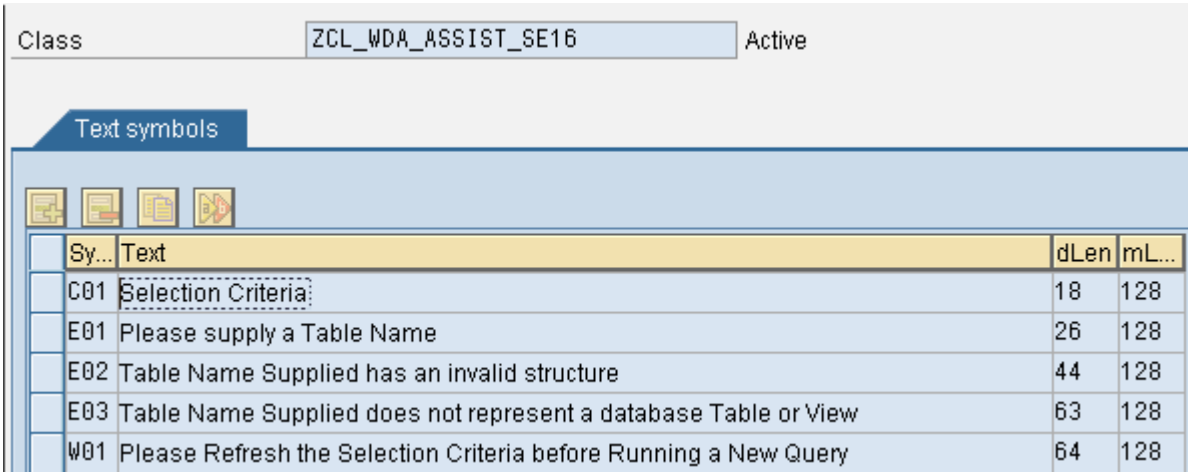


Figure 3 - Assistance Class Text Elements

Component Controller

We actually won't place any logic into the component controller or its context. Normally you would want to design your component so that much of the data is defined in the component controller context and then mapped to both the view context(s) and the component usages' interface controllers. However we have a bit of a special case here. Due to the dynamic nature of the process we will perform, we will have to create most of our context elements at runtime and then bind them dynamically.

View

Due to the relative simplicity of the UI elements that we will build at design time, I opted to place the entire UI into a single View named MAIN_VIEW.

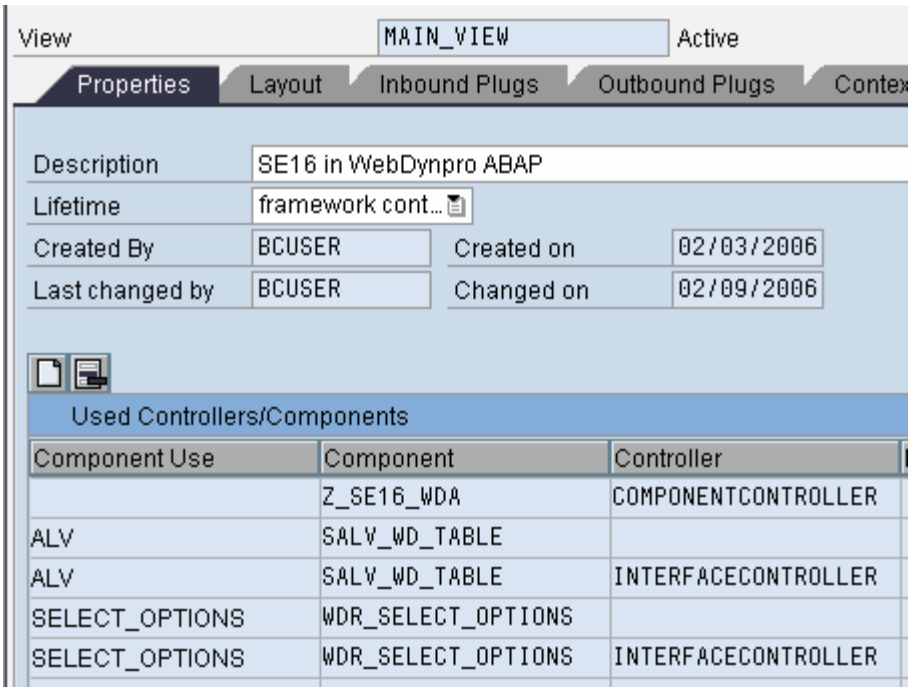


Figure 4 - View Properties

As you can see from the View Properties we will need to add Component Usages for both the ALV and SELECT_OPTIONS.

View Context

The View Context is also relatively small. We only need a handful of attributes that will map to the static UI elements. Later in code we will create the main context attributes dynamically.

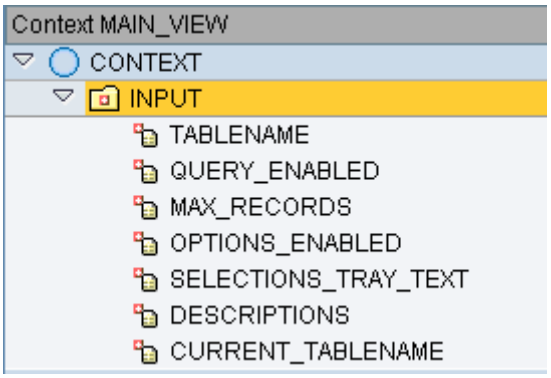


Figure 5 - View Context

The Node INPUT has a Cardinality of 1:1.

TABLENAME type DD02-DBTABNAME

QUERY_ENABLED type WDY_BOOLEAN

MAX_RECORDS type TBMAXSEL

OPTIONS_ENABLED type WDUI_VISIBILITY

SELECTIONS_TRAY_TEXT type STRING

DESCRIPTIONS type WDY_BOOLEAN

CURRENT_TABLENAME type STRING

Actions

Before we start to design our Layout, I find it useful to first create my actions. We are ultimately going to have three buttons in our UI. Each one will have a corresponding Action. When a new table name is input, we need to refresh the selection options for this we will create the action REFRESH_SEL.

We will have a section of the UI where users can change certain settings within our component. This section of the UI is hidden by default. There will be a button that will toggle the display of this section. This button will fire the action OPTIONS_SEL.

Finally once the user is ready to perform the query against the table selected and with their input criteria, they will press a button that fires the action RUN_QUERY.

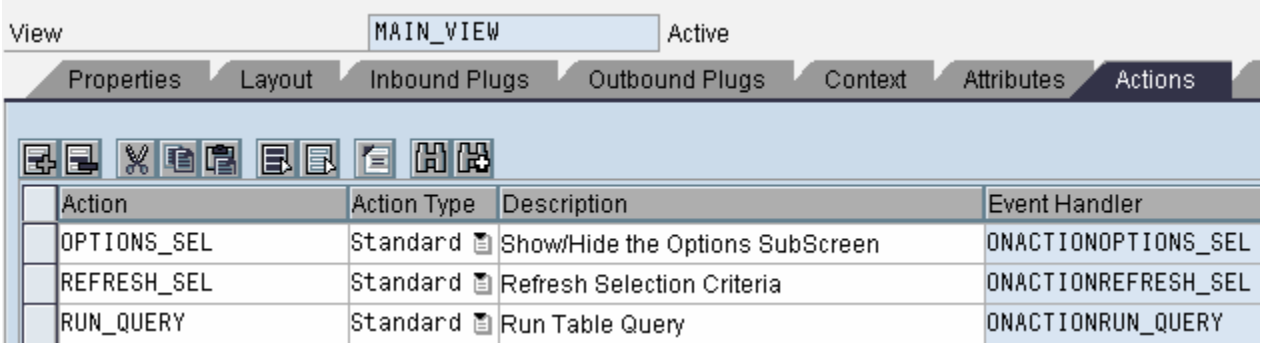


Figure 6 - View Actions

Layout

Our Layout will consist of five major areas. There will be a page header and then four trays.

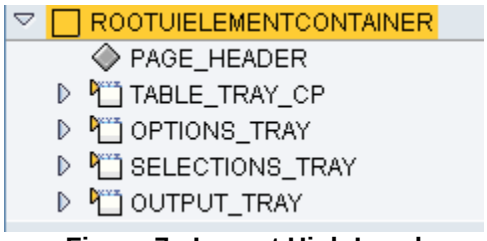


Figure 7 - Layout High Level

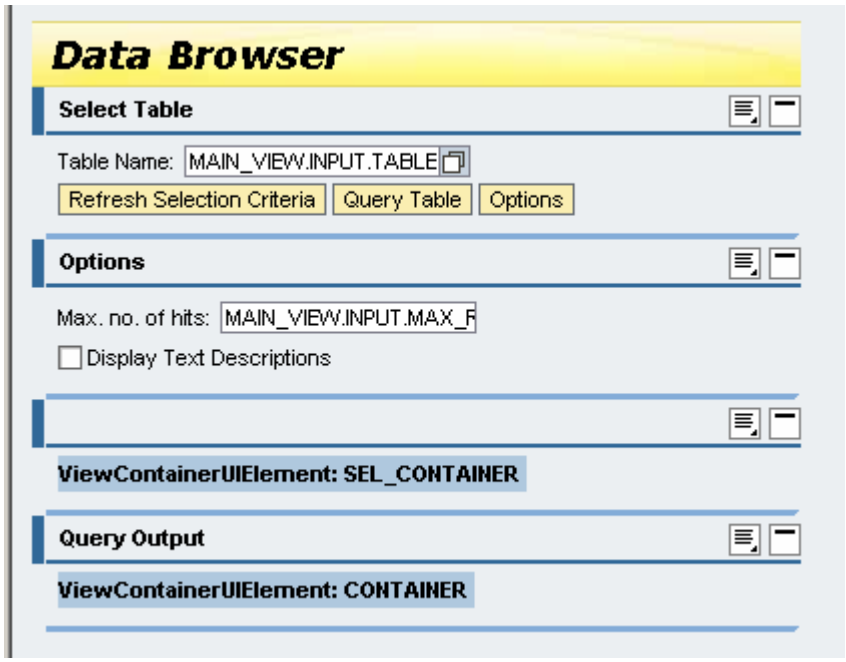


Figure 8 - Layout Graphical Overview

In the Page_Header element we will only set the property Title to “Data Browser”.

TABLE_TRAY_CP

This tray will contain area where the user can input the table name they want to query against. It will also contain all the action buttons.

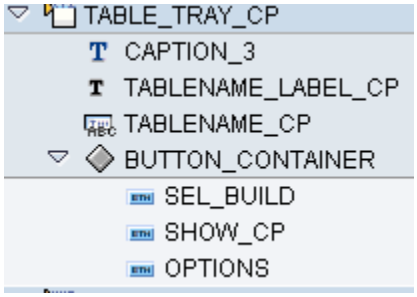
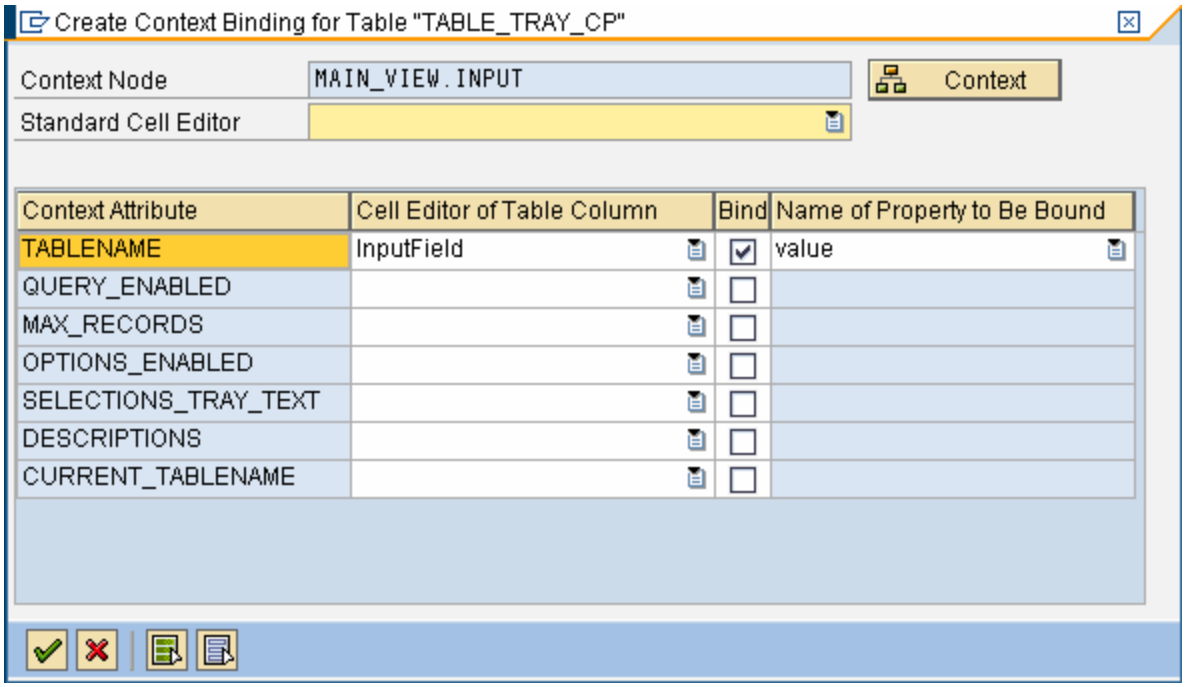


Figure 9 - TABLE_TRAY_CP UI Hierarchy

Create an Element of type Tray called TABLE_TRAY_CP. Set the Layout Property to FlowLayout. The creation of this element will also create a Caption. Set the text property of the caption to “Select Table”.

Right-click on the TABLE_TRAY_CP element and choose Create Container Form from the Context Menu. After choosing your View Context Node, Input, you can create a binding to the TABLENAME attribute.



This “Wizard” will create both the label and input field for the tablename. After the generation of these elements, we will only edit one property. Change the onAction property of the input field to REFRESH_SEL. That way if the user presses enter while the focus is on this field, it will fire the same action as pressing the Refresh Selection Criteria button; making the user interface just a little bit more friendly.

Now create a ButtonRow element with the ID BUTTON_CONTAINER.

Right-click on the ButtonRow element and choose the Insert Button. We are going to create three buttons.

The first button has the ID SEL_BUILD. It has the text "Refresh Selection Criteria" and the onAction property should be set to the action we created before called REFRESH_SEL.

The second button has the ID SHOW_CP. It has the text "Query Table" and the onAction is linked to RUN_QUERY.

The final button has the ID OPTIONS. It has the text “Options" and the onAction is linked to OPTIONS_SEL.

OPTIONS_TRAY

The options tray is an entire area of the UI that can be hidden. It contains several input elements for changing options like the maximum number of records to be returned by the query (which defaults to 500). As a long time developer I also often know the table field names better than the field descriptions. Therefore I like the option in SE16 of displaying the names instead of the descriptions. There is a checkbox that will toggle between the two.

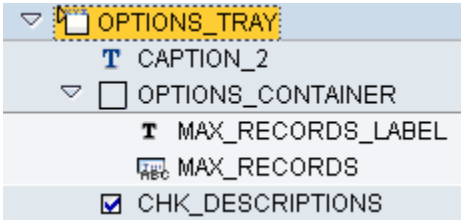


Figure 10 - OPTIONS_TRAY UI Hierarchy

Create an Element of type Tray called OPTIONS_TRAY. Set the Layout Property to MatrixLayout. Then bind the visible property to the Context attribute OPTIONS_ENABLED. Binding can be set by pressing the button to the right of the property input area.

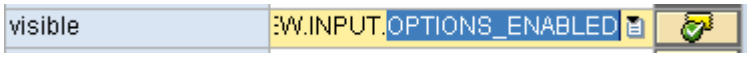


Figure 11 - Visible Binding

The creation of this element will also create a Caption. Set the text property of the caption to “Options.”

Now create a TransparentContainer element with the ID of OPTIONS_CONTAINER. Its Layout Property should be FlowLayout. Its LayoutData Property should be MatrixHeadData.

Right-click on the OPTIONS_CONTAINER and choose Create Container Form. Repeat the Process from before to create a label and input field for the context attribute MAX_RECORDS.

Right-lick on the OPTIONS_TRAY and choose Insert Element. Create a CheckBox Element with the ID of CHK_DESCRIPTIONS. Set the LayoutData to MatrixHeadData. Set the text property to ‘Display Text Descriptions’. Set the onToggle Event to the action REFRESH_SEL. Finally bind the checked property to the Context Attribute DESCRIPTIONS.

SELECTIONS_TRAY

The SELECTIONS_TRAY will hold all of our Select Options. Because they are actually contained in another Web Dynpro Component, all we will need here is a ViewContainerUIElement.

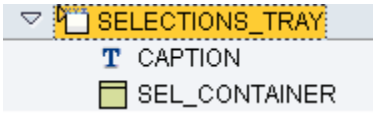


Figure 12 - SELECTIONS_TRAY UI Hierarchy

Create an Element of type Tray called SELECTIONS_TRAY. Set the Layout Property to FlowLayout.

The creation of this element will also create a Caption. Bind the text property of the caption to context attribute SELECTIONS_TRAY_TEXT.

Create a single ViewContainerUIElement with an ID of SEL_CONTAINER. Later using the Window maintenance screen we will map the external component to this container.

OUTPUT_TRAY

The OUTPUT_TRAY is very similar to the SELECTIONS_TRAY in that it will only hold a ViewContainerUIElement for the ALV.

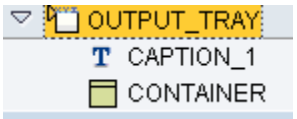


Figure 13 - OUTPUT_TRAY UI Hierarchy

Create an Element of type Tray called OUTPUT_TRAY. Set the Layout Property to FlowLayout.

The creation of this element will also create a Caption. Set the text property of the caption to 'Query Output'.

Create a single ViewContainerUIElement with an ID of CONTAINER. Later using the Window maintenance screen we will map the external component to this container.

View Attributes

There are two object references related to the Select Options Component that we will want to retain throughout the processing of our View. For these we will create the View Attributes LV_SEL_HANDLER and LV_WD_SELECTION_OPTIONS.

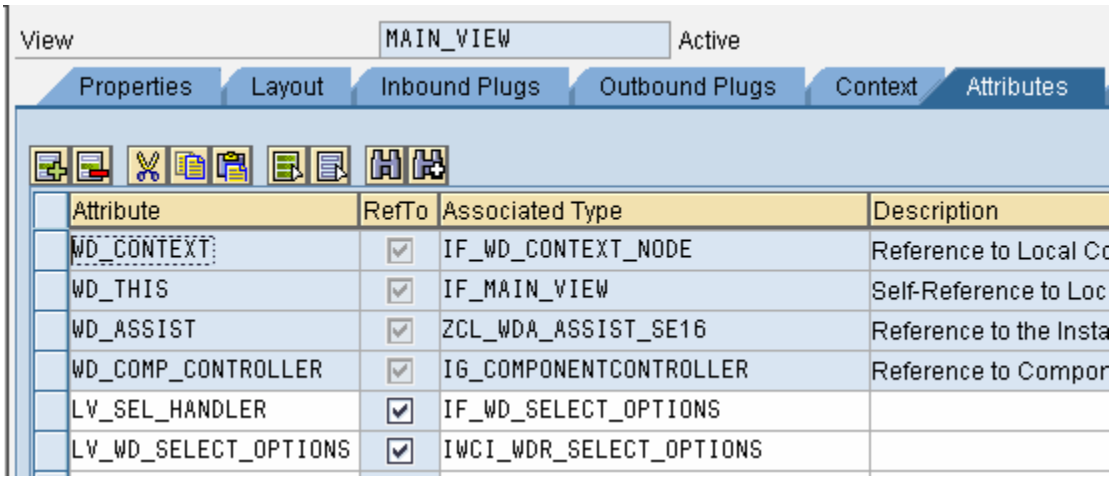


Figure 14 - View Attributes

Methods

If you now go to the Methods tab of our View you should see the default methods as well as Event Handlers for each of our Actions. In addition to these methods, we will also want to add three custom methods that will support the processing within our actions.

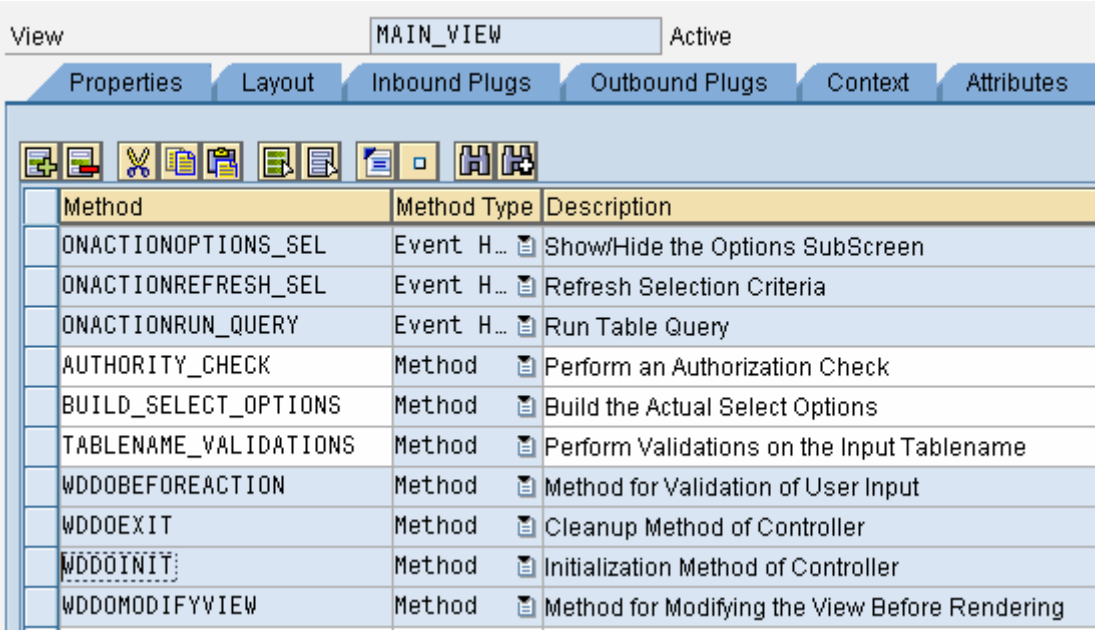


Figure 15 - View Methods

Method WDDOINIT

This method is called at the instantiation of the View. We will use it to initialize some of our context attributes. We will also initialize the Select Options.

```
method WDDOINIT .
    DATA: l_ref_cmp_usage TYPE REF TO if_wd_component_usage.

    l_ref_cmp_usage =   wd_this->wd_cpuse_select_options( ).
    IF l_ref_cmp_usage->has_active_component( ) IS INITIAL.
      l_ref_cmp_usage->create_component( ).
    ENDIF.

* get a pointer to the interface controller of the select options
*component
    wd_this->lv_wd_select_options =
         wd_this->wd_cpifc_select_options( ).

* init the select screen
    wd_this->lv_sel_handler =
         wd_this->lv_wd_select_options->init_selection_screen( ).

    DATA:
      node_input         TYPE REF TO if_wd_context_node,
      elem_input         TYPE REF TO if_wd_context_element,
      stru_input         TYPE if_main_view=>element_input ,
      item_query_enabled LIKE stru_input-query_enabled.

* navigate from <CONTEXT> to <INPUT> via lead selection
    node_input = wd_context->get_child_node( name = if_main_view=>wdctx_input ).
```

```
* get element via lead selection
    elem_input = node_input->get_element(  ).

* get single attribute
    elem_input->set_attribute(
      EXPORTING
        name =  `QUERY_ENABLED`
        value = abap_false ).

* get single attribute
    elem_input->set_attribute(
      EXPORTING
        name =  `MAX_RECORDS`
        value = `500` ).

* get single attribute
    elem_input->set_attribute(
      EXPORTING
        name =  `DESCRIPTIONS`
        value = abap_true ).

* get single attribute
    elem_input->set_attribute(
      EXPORTING
        name =  `OPTIONS_ENABLED`
        value = CL_WDL_CORE=>IF_WDL_CORE~visibility_none ).

  data: str_desc type string.
  move 'Selection Criteria'(c01) to str_desc.
    elem_input->set_attribute(
      EXPORTING
        name =  `SELECTIONS_TRAY_TEXT`
        value = str_desc ).

endmethod.
```

Method ONACTIONOPTIONS_SEL

This method is the event handler for the Action OPTIONS_SEL. It is responsible for triggering the toggle visibility for the Options Tray.

```
METHOD ONACTIONOPTIONS_SEL .
  DATA:
   node_input                          TYPE REF TO if_wd_context_node,
   elem_input                          TYPE REF TO if_wd_context_element,
   stru_input                          TYPE if_main_view=>element_input ,
   item_query_enabled                  LIKE stru_input-query_enabled.

* navigate from <CONTEXT> to <INPUT> via lead selection
  node_input = wd_context->get_child_node( name = if_main_view=>wdctx_input ).

* get element via lead selection
```

```
  elem_input = node_input->get_element(  ).

  DATA: options_enabled like cl_wdl_core=>IF_WDL_CORE~VISIBILITY.

* get single attribute
  elem_input->get_attribute(
    EXPORTING
      name =  `OPTIONS_ENABLED`
    IMPORTING
      value = options_enabled ).
  IF options_enabled = cl_wdl_core=>IF_WDL_CORE~visibility_none. "Invisible
    options_enabled = cl_wdl_core=>IF_WDL_CORE~visibility_visible. "Visible
  ELSE.
    options_enabled = cl_wdl_core=>IF_WDL_CORE~visibility_none. "Invisible
  ENDIF.

* get single attribute
  elem_input->set_attribute(
    EXPORTING
      name =  `OPTIONS_ENABLED`
      value = options_enabled ).

ENDMETHOD.
```

Method ONACTIONREFRESH_SEL

This method is the event handler for the action REFRESH_SEL. It will first check the query authority for the table in question. It will then run validations on the table name input. It will then trigger the method that builds the select options.

```
METHOD onactionrefresh_sel .

*   get message manager
  DATA: l_current_controller TYPE REF TO if_wd_controller,
        l_message_manager    TYPE REF TO if_wd_message_manager.
  l_current_controller ?= wd_this->wd_get_api( ).
  l_message_manager = l_current_controller->get_message_manager( ).

* remove all the existing Selection Screen Elements
  wd_this->lv_sel_handler->remove_all_sel_screen_items( ).

* authority check
  wd_this->authority_check( ).
  IF l_message_manager->is_empty( ) = abap_false.
    EXIT.
  ENDIF.

  DATA:
    node_input                          TYPE REF TO if_wd_context_node,
    elem_input                          TYPE REF TO if_wd_context_element,
    stru_input                          TYPE if_main_view=>element_input ,
    item_query_enabled                  LIKE stru_input-query_enabled.
* navigate from <CONTEXT> to <INPUT> via lead selection
```

```
  node_input = wd_context->get_child_node( name = if_main_view=>wdctx_input ).
* get element via lead selection
  elem_input = node_input->get_element(  ).

* get single attribute
  elem_input->set_attribute(
    EXPORTING
      name =  `QUERY_ENABLED`
      value = abap_true ).

  DATA: str_tablename TYPE string.
* get single attribute
  elem_input->get_attribute(
    EXPORTING
      name =  `TABLENAME`
    IMPORTING
      value = str_tablename ).

* Perform Table Name Validations
  wd_this->tablename_validations( tablename = str_tablename
                                  elem_input = elem_input ).
  IF l_message_manager->is_empty( ) = abap_false.
    elem_input->set_attribute(
    EXPORTING
      name =  `QUERY_ENABLED`
      value = abap_false ).
    EXIT.
  ENDIF.

* Backup the current tablename so that we check for changes later
  elem_input->set_attribute(
    EXPORTING
      name =  `CURRENT_TABLENAME`
      value = str_tablename ).

* Build the Select Options for each Field in the Table Structure
  wd_this->build_select_options(
    elem_input = elem_input
    tablename =  str_tablename ).

ENDMETHOD.
```

Method ONACTIONRUN_QUERY

This method is the event handler for the action RUN_QUERY. It contains the dynamic logic to build the data base query using the ranges returned from the select options. We use the dynamic class creation to support the various number of field symbols that we might need because of the dynamic select options.

At the end of the processing of this method, you might notice the logic to dynamically create a node in our context to bind the table results to. This node context is then also bound to the data attribute of the ALV component.

```
METHOD onactionrun_query .
  DATA:
   rootnode_info TYPE REF TO if_wd_context_node_info,
   dyn_node      TYPE REF TO if_wd_context_node,
   tabname_node  TYPE REF TO if_wd_context_node,
   current_tablename TYPE string,
   tablename         TYPE string.
*   get message manager
  DATA: l_current_controller TYPE REF TO if_wd_controller,
        l_message_manager    TYPE REF TO if_wd_message_manager.
  l_current_controller ?= wd_this->wd_get_api( ).
  l_message_manager = l_current_controller->get_message_manager( ).

  rootnode_info = wd_context->get_node_info( ).
  tabname_node = wd_context->get_child_node( name = 'INPUT' ).
  tabname_node->get_attribute( EXPORTING name = 'TABLENAME'
                               IMPORTING value = tablename ).

  tabname_node->get_attribute( EXPORTING name = 'CURRENT_TABLENAME'
                               IMPORTING value = current_tablename ).

* The user has changed the value of the input field tablename
* but they have not refreshed the selection options
* this will lead to inconsistencies and errors - so we trap it here
  IF current_tablename IS INITIAL
    OR tablename NE current_tablename.
*   report message
    l_message_manager->report_warning(
        message_text   = 'Please Refresh the Selection Criteria before Running a
 New Query'(w01)
 ).
    EXIT.
  ENDIF.

  TRANSLATE tablename TO UPPER CASE.
  DATA: s_max_records TYPE string.
  tabname_node->get_attribute( EXPORTING name = 'MAX_RECORDS'
                             IMPORTING value = s_max_records ).

*   Create dynamic sub node name TEST1 of structure (tablename)
  cl_wd_dynamic_tool=>create_nodeinfo_from_struct(
    parent_info = rootnode_info
    node_name = tablename
    structure_name = tablename
    is_multiple = abap_true ).

  DATA: stru_tab TYPE REF TO data.
  FIELD-SYMBOLS <tab> TYPE table.

*  create internal table that matches the type for our input tablename
  CREATE DATA stru_tab TYPE TABLE OF (tablename).
  ASSIGN stru_tab->* TO <tab>.
```

```
******** Begin Dynamic Coding for Selection
* This dyanamic code will create field symbols for each Select Option
* that has a valid range table.  It will then use each of these in
* the where condition of the selection
  DATA itab1  TYPE TABLE OF string.
  DATA prog  TYPE string.
  DATA class TYPE string.
  DATA code_string TYPE string.
  DATA: sel_fields TYPE if_wd_select_options=>tt_selection_screen_item.
  FIELD-SYMBOLS: <wa_fields> LIKE LINE OF sel_fields,
                 <wa_generic> TYPE table.

  wd_this->lv_sel_handler-
>get_selection_fields( IMPORTING et_fields = sel_fields ).
  APPEND `program.`                      TO itab1.
  APPEND `class zz_main_dyn definition.`       TO itab1.
  APPEND `  public section.`              TO itab1.
  APPEND `    class-methods meth `       TO itab1.
  APPEND `    importing `                TO itab1.
  APPEND `      itab type ref to data ` TO itab1.
  APPEND `      sel_fields TYPE if_wd_select_options=>tt_selection_screen_item.
` TO itab1.
  APPEND `endclass.`                     TO itab1.
  APPEND `class zz_main_dyn implementation.`   TO itab1.
  APPEND `  method meth.`                TO itab1.
  APPEND `    FIELD-SYMBOLS <table> TYPE ANY TABLE.` TO itab1.
  APPEND `    ASSIGN itab->* TO <table>.` TO itab1.
  APPEND `   field-symbols: <wa_field> like line of sel_fields.` TO itab1.

* Build the field symbols for each Range table
  LOOP AT sel_fields ASSIGNING <wa_fields>.
    ASSIGN <wa_fields>-mt_range_table->* TO <wa_generic>.
    IF <wa_generic> IS NOT INITIAL.
      CONCATENATE `field-symbols: <wa_` <wa_fields>-m_id
      `> type table.` INTO code_string.
      APPEND code_string TO itab1.
      CONCATENATE `data: rt_` <wa_fields>-m_id
      ` type ref to data.` INTO code_string.
      APPEND code_string TO itab1.
    ENDIF.
  ENDLOOP.

* Assign each Range Table to it's corresponding Field Symbol
  DATA: index TYPE string.
  LOOP AT sel_fields ASSIGNING <wa_fields>.
    ASSIGN <wa_fields>-mt_range_table->* TO <wa_generic>.
    IF <wa_generic> IS NOT INITIAL.
      MOVE sy-tabix TO index.
      CONCATENATE `read table sel_fields index ` index
                  ` assigning <wa_field>.` INTO code_string.
      APPEND code_string TO itab1.

      CONCATENATE `rt_` <wa_fields>-m_id
```

```
                  ` = <Wa_field>-mt_range_Table.` INTO code_string.
      APPEND code_string TO itab1.
      CONCATENATE `assign rt_` <wa_fields>-m_id
                  `->* to <wa_` <wa_fields>-m_id `>.` INTO code_string.
      APPEND code_string TO itab1.
    ENDIF.
  ENDLOOP.

* Build the basic part of the selection allowing for the options to set the max
records
  CONCATENATE `select * from ` tablename
              ` into table <table> `
              ` up to ` s_max_records ` rows `
              INTO code_string.
  APPEND code_string TO itab1.

* Build the where clause from the range table's field symbols
  DATA: first_pass TYPE boolean VALUE abap_true.
  LOOP AT sel_fields ASSIGNING <wa_fields>.
    ASSIGN <wa_fields>-mt_range_table->* TO <wa_generic>.
    IF <wa_generic> IS NOT INITIAL.
      IF first_pass = abap_true.
        first_pass = abap_false.
        MOVE ` where ` TO code_string.
      ELSE.
        MOVE ` and ` TO code_string.
      ENDIF.
      CONCATENATE code_string <wa_fields>-m_id ` in <wa_`
                  <wa_fields>-m_id `> ` INTO code_string.
      APPEND code_string TO itab1.
    ENDIF.
  ENDLOOP.
  MOVE '.' TO code_string.
  APPEND code_string TO itab1.
  APPEND `  endmethod.` TO itab1.
  APPEND `endclass.`                    TO itab1.

* Compile the dynamic Class and check for generation errors
  DATA: mess TYPE string.
  GENERATE SUBROUTINE POOL itab1 NAME prog MESSAGE mess.
* I debated if I wanted to display generation errors
* They likely won't make any sense to an end user -
* they are really meant for serious troubleshooting
* In the end I thought it was better to throw this error to the screen
* rather than dump when the class
* attempts to be called.
  IF mess IS NOT INITIAL.
*   report message
    l_message_manager->report_error_message(
        message_text   = mess ).
    EXIT.
  ENDIF.
```

```
* Prepare the parameters for the dynamic class call
  CONCATENATE `\PROGRAM=` prog `\CLASS=ZZ_MAIN_DYN` INTO class.
  DATA: ptab TYPE abap_parmbind_tab,
        ptab_line TYPE abap_parmbind.
  ptab_line-name = 'ITAB'.
  ptab_line-kind = cl_abap_objectdescr=>exporting.
  GET REFERENCE OF stru_tab INTO ptab_line-value.
  INSERT ptab_line INTO TABLE ptab.

  ptab_line-name = 'SEL_FIELDS'.
  ptab_line-kind = cl_abap_objectdescr=>exporting.
  GET REFERENCE OF sel_fields INTO ptab_line-value.
  INSERT ptab_line INTO TABLE ptab.

* Call the Dynamic Class
  CALL METHOD (class)=>meth
    PARAMETER-TABLE
      ptab.

********End Dynamic Coding

*  get instance of new node
  dyn_node = wd_context->get_child_node( name = tablename ).

*  Bind internal table to context node
  dyn_node->bind_table( <tab> ).

* Connect to the component Usage of the ALV
  DATA: l_ref_cmp_usage TYPE REF TO if_wd_component_usage.
  l_ref_cmp_usage =   wd_this->wd_cpuse_alv( ).
  IF l_ref_cmp_usage->has_active_component( ) IS INITIAL.
    l_ref_cmp_usage->create_component( ).
  ENDIF.

* Through the interface controller of the ALV Component set the DATA node dynami
cally
  DATA: l_ref_interfacecontroller TYPE REF TO iwci_salv_wd_table .
  l_ref_interfacecontroller =   wd_this->wd_cpifc_alv( ).
  l_ref_interfacecontroller->set_data(
    r_node_data = dyn_node  ).

ENDMETHOD.
```

Method AUTHORITY_CHECK

This method performs the actual authorization check and issues a message to the message manager via a T100 Message. This has a very nice output.

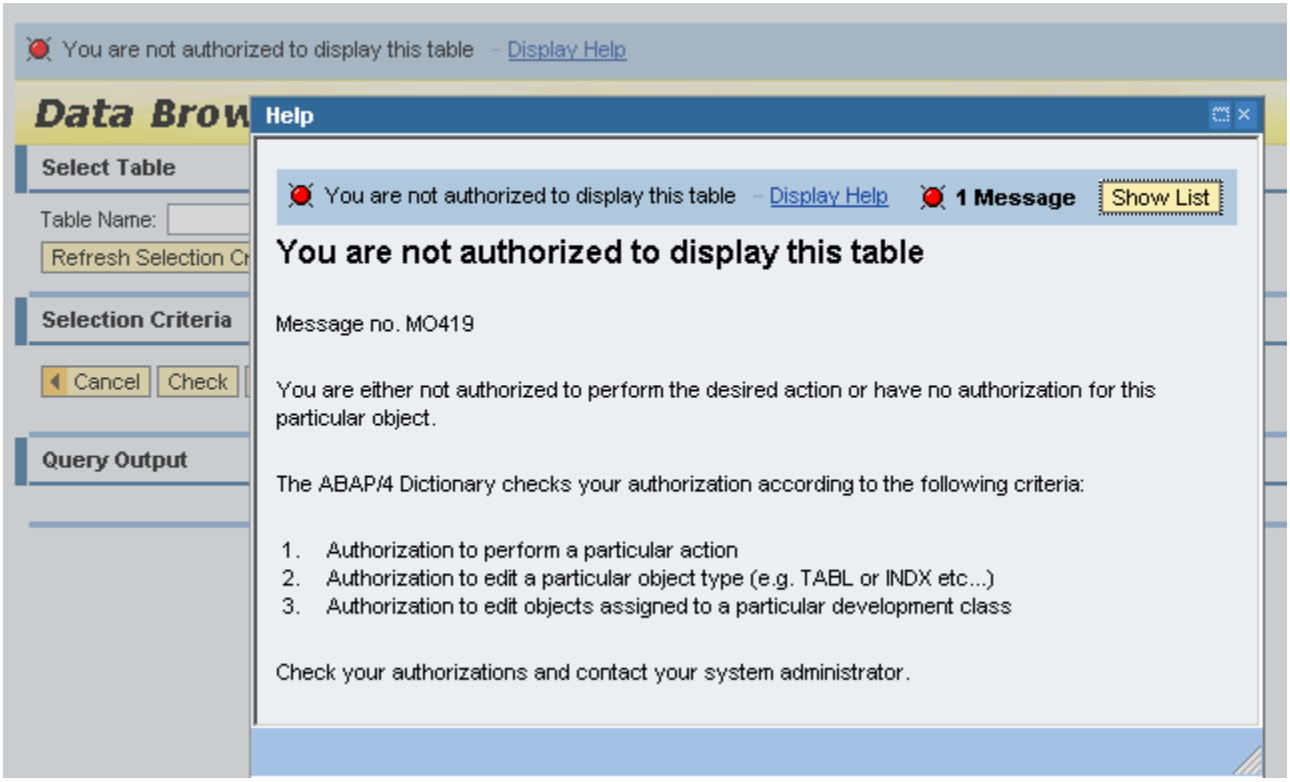


Figure 16 - Error Message Output Example

```
METHOD authority_check .
  DATA:
    node_input                          TYPE REF TO if_wd_context_node,
    elem_input                          TYPE REF TO if_wd_context_element,
    stru_input                          TYPE if_main_view=>element_input ,
    item_tablename                      LIKE stru_input-tablename.
*   navigate from <CONTEXT> to <INPUT> via lead selection
  node_input = wd_context->get_child_node( name = if_main_view=>wdctx_input ).

*   get element via lead selection
  elem_input = node_input->get_element(  ).

*   get single attribute
  elem_input->get_attribute(
    EXPORTING
      name =  `TABLENAME`
    IMPORTING
      value = item_tablename ).


  DATA: l_tddat TYPE tddat.
  SELECT SINGLE * FROM tddat INTO l_tddat
   WHERE tabname = item_tablename.
  IF sy-subrc <> 0  OR  l_tddat-cclass = space.
    l_tddat-cclass = '&NC&'.              " 'non classified table'
```

```
  ENDIF.

  AUTHORITY-CHECK OBJECT 'S_TABU_DIS'
           ID 'DICBERCLS' FIELD l_tddat-cclass
           ID 'ACTVT' FIELD '03'.
  IF sy-subrc NE 0.
*   get message manager
    DATA: l_current_controller TYPE REF TO if_wd_controller,
          l_message_manager    TYPE REF TO if_wd_message_manager.
    l_current_controller ?= wd_this->wd_get_api( ).
    l_message_manager = l_current_controller->get_message_manager( ).

* get single attribute
    elem_input->set_attribute(
      EXPORTING
        name =  `QUERY_ENABLED`
        value = abap_false ).

    DATA: str_tablename TYPE string.
* get single attribute
    elem_input->set_attribute(
      EXPORTING
        name =  `TABLENAME`
        value = `` ).

    elem_input->set_attribute(
      EXPORTING
        name =  `CURRENT_TABLENAME`
        value = `` ).

* report message
    l_message_manager->report_t100_message(
        msgid         = 'MO'
        msgno         = '419'
        msgty         = 'E'  ).

  ENDIF.
ENDMETHOD.
```

Method BUILD_SELECT_OPTIONS

This method is responsible for using the RTTS to read all the fields in the table we are going to query. It will then create select option UI elements for each of them.

We have several importing parameters into this method.

Method BUILD_SELECT_OPTIONS

Parameter	Type	RefTo	Associated Type
ELEM_INPUT	Importing	☑	IF_WD_CONTEXT_ELEMENT
TABLENAME	Importing	☐	STRING

Figure 17 - BUILD_SELECT_OPTIONS Parameters

```
METHOD build_select_options .
  DATA: lt_range_table TYPE REF TO data,
      rt_range_table TYPE REF TO data,
      read_only      TYPE abap_bool,
      descriptions   TYPE abap_bool,
      typename       TYPE string.

* get single attribute
  elem_input->get_attribute(
    EXPORTING
      name =  `DESCRIPTIONS`
    IMPORTING
      value = descriptions ).

* Query the RTTS for the structure of the Input Table
  DATA: descriptor TYPE REF TO cl_abap_structdescr.
  descriptor ?= cl_abap_structdescr=>describe_by_name( tablename ).
  DATA: flddescr TYPE ddfields.
  flddescr = descriptor->get_ddic_field_list( ).

* get the structure Description
  DATA: rel_name TYPE string.
  DATA: str_desc TYPE string.
  DATA: l_dd07l  TYPE dd07l.
  rel_name = descriptor->get_relative_name( ).
  SELECT SINGLE ddtext FROM dd02t INTO str_desc
          WHERE tabname = rel_name
            AND ddlanguage = sy-langu
            AND as4local = 'A'.
  IF sy-subrc NE 0.
    SELECT SINGLE ddtext FROM dd02t INTO str_desc
         WHERE tabname = rel_name
           AND ddlanguage = 'E'
           AND as4local = 'A'.
  ENDIF.
  CONCATENATE 'Selection Criteria'(c01)
              ` - ` str_desc INTO str_desc.
  elem_input->set_attribute(
    EXPORTING
      name =  `SELECTIONS_TRAY_TEXT`
      value = str_desc ).

* Loop through the Structure of the Table and create Ranges
* and Select Options for each field
```

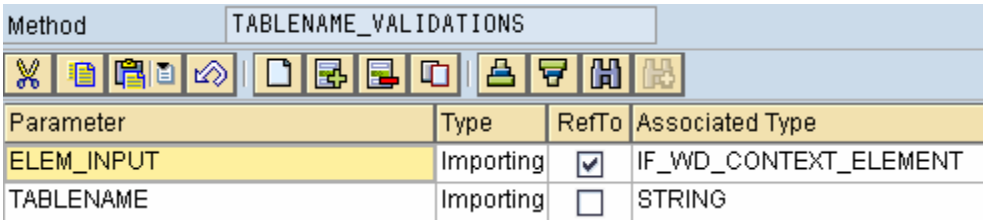
```
  FIELD-SYMBOLS: <wa_def>   LIKE LINE OF flddescr.
  DATA: l_typename  TYPE string,
        l_fieldname TYPE string.
  LOOP AT flddescr ASSIGNING <wa_def> WHERE fieldname  NE 'MANDT'.
    l_typename = <wa_def>-rollname.
    l_fieldname = <wa_def>-fieldname.
* create a range table that consists of this new data element
    lt_range_table =
      wd_this->lv_sel_handler->create_range_table(
           i_typename = l_typename ).

    IF descriptions = abap_true.
* add a new field to the selection
      wd_this->lv_sel_handler->add_selection_field(
                      i_id = l_fieldname
                      it_result = lt_range_table
                      i_read_only = read_only ).
    ELSE.
      wd_this->lv_sel_handler->add_selection_field(
                 i_id = l_fieldname
                 i_description = l_fieldname
                 it_result = lt_range_table
                 i_read_only = read_only ).
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

Method TABLENAME_VALIDATION

This final method will perform all of our validations on the table name that the user has input.

We have several importing parameters into this method.



Method TABLENAME_VALIDATIONS

Parameter	Type	RefTo	Associated Type
ELEM_INPUT	Importing	☑	IF_WD_CONTEXT_ELEMENT
TABLENAME	Importing	☐	STRING

Figure 18 - TABLENAME_VALIDATIONS Parameters

```
METHOD tablename_validations .

*   get message manager
  DATA: l_current_controller TYPE REF TO if_wd_controller,
        l_message_manager    TYPE REF TO if_wd_message_manager.
  l_current_controller ?= wd_this->wd_get_api( ).
  l_message_manager = l_current_controller->get_message_manager( ).

* test for a supplied table name
  IF tablename IS INITIAL.
*   report message
```

```
    l_message_manager->report_attribute_error_message(
        message_text   = 'Please supply a Table Name'(e01)
        element        = elem_input
        attribute_name = 'TABLENAME' ).
    EXIT.
  ENDIF.

* Test for a Valid Database Table or View
  DATA: l_tabname TYPE tabname.
  SELECT SINGLE tabname FROM dd02l INTO l_tabname
          WHERE tabname = tablename
            AND as4local = 'A'.
  IF sy-subrc NE 0.
    l_message_manager->report_attribute_error_message(
        message_text   = 'Table Name Supplied does not represent a database Tabl
e or View'(e03)
        element        = elem_input
        attribute_name = 'TABLENAME' ).
    EXIT.
  ENDIF.

* Read the field listing for the Table Supplied
  DATA: descriptor TYPE REF TO cl_abap_structdescr.
  descriptor ?= cl_abap_structdescr=>describe_by_name( tablename ).
  DATA: flddescr TYPE ddfields.
  flddescr = descriptor->get_ddic_field_list( ).
  IF flddescr IS INITIAL.
*   report message
    l_message_manager->report_attribute_error_message(
        message_text   = 'Table Name Supplied has an invalid structure'(e02)
        element        = elem_input
        attribute_name = 'TABLENAME' ).
    EXIT.
  ENDIF.
ENDMETHOD.
```

Windows

In the Window maintenance screen, we need to assign our single view to the MAIN window. If you then expand the nodes under the main window, you will see our two containers. If you right-click on each of the container names you can choose Embed View from the Context Menu. In the CONTAINER element embed the ALV TABLE View. In the SEL_CONTAINER element embed the SELECT_OPTIONS WND_SELECTION_SCREEN View.

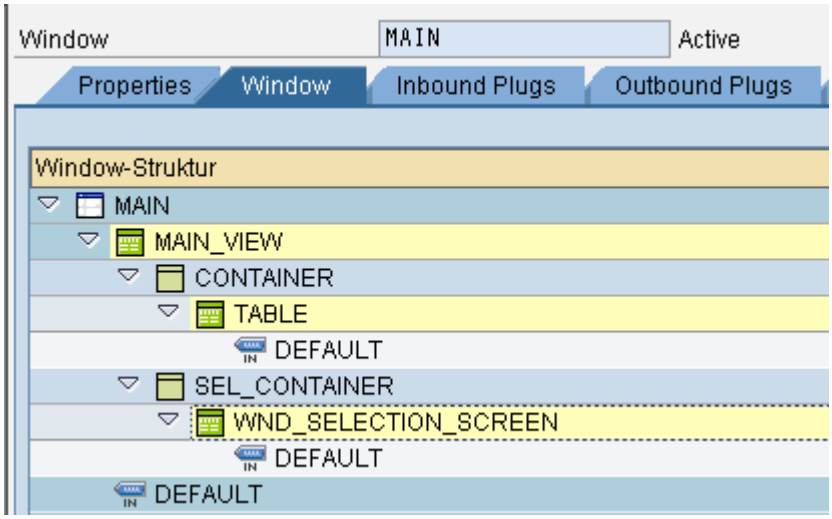


Figure 19 - Window Maintenance

Web Dynpro Application

As the final step, you can right-click on the component name in the SE80 Object Hierarchy and choose Create -> Web Dynpro Application.

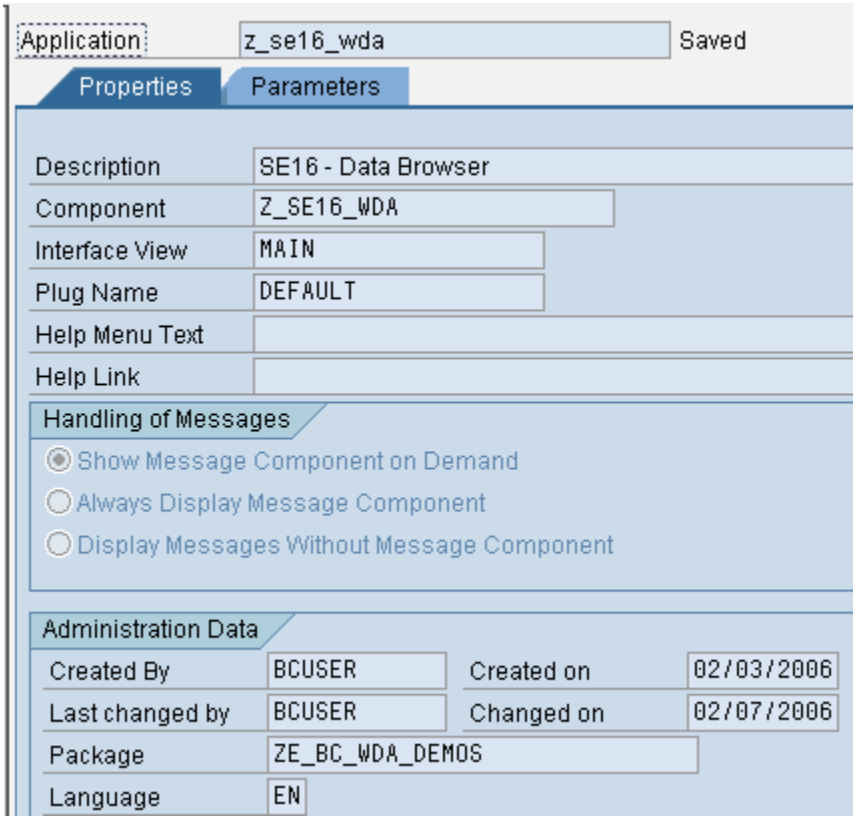


Figure 20 - Web Dynpro Application

Author Bio

Thomas Jung is an SAP Netweaver Product Manager focusing on the development aspects across Enterprise Information Management. Before joining SAP Labs in 2006, Thomas was an applications developer for an SAP customer company. He was involved in SAP implementations at this customer as an ABAP developer for nearly 10 years. He is also the co-author of the SAP Press Book "Advanced BSP Programming."

Disclaimer & Liability Notice

This document may discuss sample coding or other information that does not include SAP official interfaces and therefore is not supported by SAP. Changes made based on this information are not supported and can be overwritten during an upgrade.

SAP will not be held liable for any damages caused by using or misusing the information, code or methods suggested in this document, and anyone using these methods does so at his/her own risk.

SAP offers no guarantees and assumes no responsibility or liability of any type with respect to the content of this technical article or code sample, including any liability resulting from incompatibility between the content within this document and the materials and services offered by SAP. You agree that you will not hold, or seek to hold, SAP responsible or liable with respect to the content of this document.